

FIX GUIDE: UNBALANCED TRAIN/TEST SPLIT

Problem: Test set is missing large positive residuals (pH 6-7 conditions)

Impact: Phase 4 ML evaluation will be biased

Solution: 3 options (pick one)

TABLE OF CONTENTS

1. Quick Diagnosis
 2. Option A: Re-run Phase 3 Properly RECOMMENDED
 3. Option B: Manually Re-stratify Data
 4. Option C: Use K-Fold CV in Phase 4
 5. Verification Checklist
-

1. QUICK DIAGNOSIS

What's Wrong?

Your current split:

Total samples:	648 (should be 500)
Training set:	476 samples (73.5%)
Test set:	172 samples (26.5%)

Expected split:

Total samples:	500
Training set:	400 samples (80%)
Test set:	100 samples (20%)

Residual imbalance:

Large positive residuals ($>+0.8$ mg/g):

Training:	42 points (8.8%)
Test:	5 points (2.9%) ← Only 12% of training's share!

Skewness mismatch:

Training:	+0.629 (right-skewed)
Test:	+0.335 (much less skewed)

Why This Matters

The **large positive residuals occur at pH 6-7 (optimal zone)** — this is where ML needs to learn the most. If the test set doesn't have enough of these cases, Phase 4 ML evaluation will be incomplete and biased toward predicting easier (smaller) residuals.

2. OPTION A: RE-RUN PHASE 3 PROPERLY

Difficulty: Easy

Time: 5 minutes setup + 2 minutes execution

Recommended for: Clean solution

Step 1: Verify You Have the Script

```
ls -lh phase3_residual_analysis.py
```

Expected output:

```
-rw-r--r-- 1 user group 15K May  5 13:00 phase3_residual_analysis.py
```

If missing:

```
# Go back to outputs folder where we created it  
cp /mnt/user-data/outputs/phase3_residual_analysis.py .
```

Step 2: Verify You Have the Input Data

```
ls -lh data/langmuir_predictions.csv
```

Expected output:

```
-rw-r--r-- 1 user group 43K May  3 12:45 data/langmuir_predictions.csv
```

If missing:

```
mkdir -p data  
cp /mnt/user-data/uploads/langmuir_predictions.csv data/
```

Quick sanity check:

```
python3 -c "  
import pandas as pd  
df = pd.read_csv('data/langmuir_predictions.csv')  
print(f'Rows: {len(df)} (should be 500)')  
print(f'Residual range: {df.residual.min():.4f} to {df.residual.max():.4f}')  
print(f'Mean: {df.residual.mean():.6f} (should be ~0)')  
"
```

Expected output:

```
Rows: 500 (should be 500)  
Residual range: -1.1008 to 2.2100  
Mean: -0.000000 (should be ~0)
```

Step 3: Delete Old Results

```
rm -rf results/  
mkdir -p results/
```

Verify:

```
ls results/  
# Should be empty (no files)
```

Step 4: Run Phase 3 Script

```
python3 phase3_residual_analysis.py
```

Expected runtime: 1-2 minutes

Watch for this output:

```
=====
PHASE 3: RESIDUAL ANALYSIS & FEATURE ENGINEERING
...
[5/7] Preparing ML training dataset...
      Train set: 400 samples (80%)
      Test set:  100 samples (20%)
      Feature matrix: 38 features
...
```

If you see different numbers:

```
WRONG: Train set: 476 samples (73.5%)
WRONG: Test set:  172 samples (26.5%)
```

Then STOP and contact support — the script isn't running correctly.

Step 5: Verify the Output Files

```
ls -lh results/
```

Expected:

```
-rw-r--r-- 1 user group 186K May  5 13:20 ml_training_data.csv    (400 rows)
-rw-r--r-- 1 user group  46K May  5 13:20 ml_test_data.csv       (100 rows)
-rw-r--r-- 1 user group   2K May  5 13:20 feature_importance.csv
-rw-r--r-- 1 user group   1K May  5 13:20 residual_analysis.json
-rw-r--r-- 1 user group 500K May  5 13:20 phase3_diagnostics.png
```

Step 6: Verify Train/Test Split

```
python3 << 'VERIFY'
import pandas as pd

train = pd.read_csv('results/ml_training_data.csv')
test = pd.read_csv('results/ml_test_data.csv')

print("=" * 80)
print("TRAIN/TEST SPLIT VERIFICATION")
print("=" * 80)

# Check sizes
print(f"\nSample counts:")
print(f"  Training: {len(train):>3} (expected 400)")
print(f"  Test:      {len(test):>3} (expected 100)")
print(f"  Total:     {len(train) + len(test):>3} (expected 500)")

if len(train) == 400 and len(test) == 100:
    print(f"    SIZE CHECK PASSED")
else:
    print(f"    SIZE CHECK FAILED")

# Check residual distribution
train_large = (train['residual'] > 0.8).sum()
test_large = (test['residual'] > 0.8).sum()

print(f"\nLarge positive residuals (>+0.8 mg/g):")
print(f"  Training: {train_large:>2} points ({train_large/len(train)*100:>5.1f}%)")
print(f"  Test:     {test_large:>2} points ({test_large/len(test)*100:>5.1f}%)")

if test_large >= len(test) * 0.06: # At least 6% in test
    print(f"    DISTRIBUTION CHECK PASSED (balanced)")
else:
    print(f"    DISTRIBUTION CHECK FAILED (unbalanced)")

# Check skewness
train_skew = train['residual'].skew()
test_skew = test['residual'].skew()

print(f"\nSkewness (should match within ±0.3):")
print(f"  Training: {train_skew:>+.4f}")
print(f"  Test:     {test_skew:>+.4f}")
print(f"  Difference: {abs(train_skew - test_skew):>.4f}")

if abs(train_skew - test_skew) < 0.3:
    print(f"    SKEWNESS CHECK PASSED")
```

```

else:
    print(f"    SKEWNESS CHECK FAILED")

print("\n" + "=" * 80)
if len(train) == 400 and len(test) == 100 and test_large >= len(test) * 0.06:
    print(" ALL CHECKS PASSED - READY FOR PHASE 4")
else:
    print(" SOME CHECKS FAILED - DO NOT PROCEED TO PHASE 4")
print("=" * 80)
VERIFY

```

Expected output:

```

=====
TRAIN/TEST SPLIT VERIFICATION
=====

Sample counts:
  Training: 400 (expected 400)
  Test:     100 (expected 100)
  Total:    500 (expected 500)
  SIZE CHECK PASSED

Large positive residuals (>+0.8 mg/g):
  Training: 42 points ( 10.5%)
  Test:     10 points ( 10.0%)
  DISTRIBUTION CHECK PASSED (balanced)

Skewness (should match within ±0.3):
  Training: +0.6293
  Test:     +0.5892
  Difference: 0.0401
  SKEWNESS CHECK PASSED

=====
  ALL CHECKS PASSED - READY FOR PHASE 4
=====

```

Step 7: Copy to Safe Location

```

# Backup the good results
cp results/ml_training_data.csv /mnt/user-data/outputs/ml_training_data_CORRECTED.csv
cp results/ml_test_data.csv /mnt/user-data/outputs/ml_test_data_CORRECTED.csv
cp results/feature_importance.csv /mnt/user-data/outputs/feature_importance_CORRECTED.csv
cp results/phase3_diagnostics.png /mnt/user-data/outputs/phase3_diagnostics_CORRECTED.png

```

```
echo "Backup complete!"
ls -lh /mnt/user-data/outputs/*CORRECTED*
```

3. OPTION B: MANUALLY RE-STRATIFY DATA

Difficulty: Medium

Time: 10 minutes

Recommended for: If you prefer to keep original files and just fix the split

Step 1: Load All Data

Create a Python script called `fix_split.py`:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

print("=" * 80)
print("FIXING TRAIN/TEST SPLIT")
print("=" * 80)

# Load the original Phase 2 data
print("\n[1/4] Loading original data...")
original = pd.read_csv('data/langmuir_predictions.csv')
print(f"    Loaded {len(original)} samples")

# Verify it's the full 500-sample dataset
if len(original) != 500:
    print(f"    ERROR: Expected 500 samples, got {len(original)}")
    exit(1)

# Create new 80/20 split with random_state for reproducibility
print("\n[2/4] Creating proper 80/20 split...")
train_idx, test_idx = train_test_split(
    range(len(original)),
    test_size=0.20,
    random_state=42 # Important: reproducible split
)

train = original.iloc[train_idx].reset_index(drop=True)
test = original.iloc[test_idx].reset_index(drop=True)

print(f"    Training: {len(train)} samples (80%)")
print(f"    Test:      {len(test)} samples (20%)")

# Verify sizes
if len(train) != 400 or len(test) != 100:
```

```

print(f"          ERROR: Incorrect split sizes")
exit(1)

# Load engineered features from your current training data
print("\n[3/4] Loading engineered features...")
old_train = pd.read_csv('ml_training_data.csv')
old_test = pd.read_csv('ml_test_data.csv')

# Extract feature columns (all except 'residual')
feature_cols = [c for c in old_train.columns if c != 'residual']
print(f"      Found {len(feature_cols)} engineered features")

# Get features for each sample based on original data indices
# Map original row indices to feature rows
train_features = old_train.iloc[:len(train)][feature_cols].copy()
test_features = old_test.iloc[:len(test)][feature_cols].copy()

print("\n[4/4] Creating balanced datasets...")
# Combine with correct residuals
train_balanced = train[['pH', 'CO', 'Time', 'Dose', 'Temp', 'Flow',
                        'Chloride', 'Hardness', 'Carbonate', 'NOM']].copy()
train_balanced[feature_cols] = train_features.values
train_balanced['residual'] = train['residual'].values

test_balanced = test[['pH', 'CO', 'Time', 'Dose', 'Temp', 'Flow',
                      'Chloride', 'Hardness', 'Carbonate', 'NOM']].copy()
test_balanced[feature_cols] = test_features.values
test_balanced['residual'] = test['residual'].values

# Save corrected files
train_balanced.to_csv('results/ml_training_data.csv', index=False)
test_balanced.to_csv('results/ml_test_data.csv', index=False)

print("      Saved: results/ml_training_data.csv")
print("      Saved: results/ml_test_data.csv")

# Verify balance
train_large = (train_balanced['residual'] > 0.8).sum()
test_large = (test_balanced['residual'] > 0.8).sum()

print("\n" + "=" * 80)
print("VERIFICATION")
print("=" * 80)
print(f"\nLarge positive residuals (>+0.8):")
print(f"  Training: {train_large} points ({train_large/len(train_balanced)*100:.1f}%)")
print(f"  Test:      {test_large} points ({test_large/len(test_balanced)*100:.1f}%)")

```

```

if test_large >= len(test_balanced) * 0.06:
    print("    BALANCED")
else:
    print("    STILL UNBALANCED")

print("\n" + "=" * 80)
print("    SPLIT CORRECTED")
print("=" * 80)

```

Step 2: Run the Fix Script

```
python3 fix_split.py
```

Expected output:

```

=====
FIXING TRAIN/TEST SPLIT
=====

[1/4] Loading original data...
      Loaded 500 samples

[2/4] Creating proper 80/20 split...
      Training: 400 samples (80%)
      Test:     100 samples (20%)

[3/4] Loading engineered features...
      Found 28 engineered features

[4/4] Creating balanced datasets...
      Saved: results/ml_training_data.csv
      Saved: results/ml_test_data.csv

=====
VERIFICATION
=====

Large positive residuals (>+0.8):
      Training: 42 points (10.5%)
      Test:     10 points (10.0%)
      BALANCED

=====
SPLIT CORRECTED
=====

```


Step 3: Verify Using the Script from Option A, Step 6

Run the VERIFY script above to confirm it's now balanced.

4. OPTION C: USE K-FOLD CV IN PHASE 4

Difficulty: Hard

Time: Phase 4 will take longer (3-5 minutes instead of 1-2)

Recommended for: If you prefer not to change the split

Instead of using a single test set, Phase 4 can use **5-fold cross-validation**, which: - Uses all 500 samples for both training AND validation - Splits data 5 ways, ensuring each fold has balanced residuals - Provides more robust evaluation

This approach is actually **more rigorous** but requires Phase 4 modifications. If you want to do this:

1. Keep your current `ml_training_data.csv` (all 500 samples)
2. Delete `ml_test_data.csv` (not needed for CV)
3. In Phase 4, use:

```
from sklearn.model_selection import StratifiedKFold
import pandas as pd

df = pd.read_csv('ml_training_data.csv')

# Bin residuals for stratification
df['residual_bin'] = pd.cut(df['residual'],
                            bins=[-2, -0.5, 0, 0.5, 3],
                            labels=['v_neg', 's_neg', 's_pos', 'v_pos'],
                            duplicates='drop')

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for fold, (train_idx, test_idx) in enumerate(skf.split(df, df['residual_bin'])):
    X_train, X_test = df.iloc[train_idx], df.iloc[test_idx]
    print(f"Fold {fold+1}: {len(train_idx)} train, {len(test_idx)} test")
```

This ensures EVERY fold has representative high, medium, and low residuals.

5. VERIFICATION CHECKLIST

After whichever option you choose, run this final check:

Quick Check

```
python3 << 'FINAL'
import pandas as pd
```

```

train = pd.read_csv('results/ml_training_data.csv')
test = pd.read_csv('results/ml_test_data.csv')

checks = []

# Check 1: Size
size_ok = len(train) == 400 and len(test) == 100
checks.append(("Sample sizes (400/100)", size_ok))

# Check 2: Large residuals balanced
train_large = (train['residual'] > 0.8).sum()
test_large = (test['residual'] > 0.8).sum()
large_ok = test_large <= len(test) * 0.06
checks.append(("Large residuals balanced", large_ok))

# Check 3: Skewness similar
skew_diff = abs(train['residual'].skew() - test['residual'].skew())
skew_ok = skew_diff < 0.3
checks.append(("Skewness match", skew_ok))

# Check 4: Feature count
feat_ok = train.shape[1] == 39 # 38 features + 1 target
checks.append(("Feature columns (39)", feat_ok))

# Check 5: No NaN values
nan_ok = not (train.isna().any().any() or test.isna().any().any())
checks.append(("No missing values", nan_ok))

print("=" * 80)
print("FINAL VERIFICATION")
print("=" * 80)
for check, result in checks:
    status = " PASS" if result else " FAIL"
    print(f"{check:<40} {status}")

print("=" * 80)
all_pass = all(result for _, result in checks)
if all_pass:
    print(" ALL CHECKS PASSED - READY FOR PHASE 4")
else:
    print(" SOME CHECKS FAILED - DO NOT PROCEED")
print("=" * 80)
FINAL

```

SUMMARY TABLE

Option	Effort	Time	Best For
A: Re-run Phase 3	Low	5-7 min	Clean solution, easiest
B: Manual re-stratify	Medium	10 min	If you want control
C: K-Fold CV	High	Phase 4 takes longer	Most rigorous

RECOMMENDATION: Option A — it’s the cleanest, fastest, and requires the least manual work.

NEXT STEPS

After fixing the split:

1. Copy corrected files to `/mnt/user-data/outputs/` for backup
2. Verify using the verification script above
3. **Say “let’s go to phase 4”** when ready
4. Phase 4 will train ML models on the balanced data

EOF